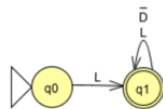


Revisão Linguagens Formais

Discussão (2021/1)

1. Escreva um Autômato Finito (AF) e uma Expressão Regular (ER) para cada um dos casos abaixo:
- a. identificadores são formados por uma letra seguido de letras, dígitos ou o caracter " _ "



$$L(L + D + _)^*$$

$L = 'a'..z, A..Z$

$D = '0'..'9'$

*praticamente todas linguagens modernas fornecem algum mecanismo de 'pattern matching' utilizando ERs, porém existe uma variedade enorme de formatos, ver por exemplo: <https://regex101.com/>,

Na ferramenta léxica que vamos utilizar: $[a-zA-Z][a-zA-Z0-9_]^*$

--> [..] - conjunto de caracteres, a-z todos os caracteres no intervalo de 'a' a 'z'

- b. números inteiros sem sinal

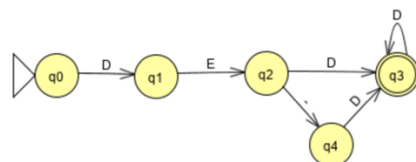


$$DD^*$$

$p: f^*(q_0, '123') \Rightarrow f^*(q_1, 23)$

$[0-9]^+$ --> + é o fecho positivo, um ou mais

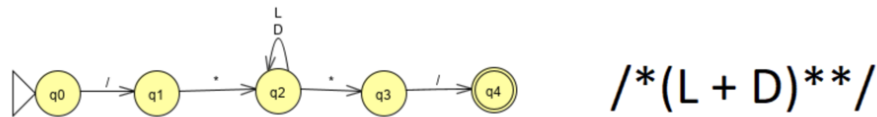
- c. números em notação científica



$$DE(D + -D)D^*$$

$123E-41$ --> $[0-9]^+[Ee](\backslash+|\backslash-)?[0-9]^+$

- d. comentários: delimitados pela sequência `"/*"` e `"*/"`



Questão – comentários aninhados são LR?

2. Qual a relação entre estes dois tipos de representação (AF e ER)?

Um AF modela a linguagem (regular) aceita por uma ER. E vice-versa.

Expressões regulares (e também gramáticas regulares) definem todo conjunto de expressões válidas (**GERADOR**) para uma linguagem (regular), enquanto os AF reconhecem (**RECONHECEDOR**) se um programa pertence a esta linguagem.

3. Defina alfabeto, sentença e linguagem.

Alfabeto: é um conjunto finito de Símbolos.

Sentença ou programa: é uma sequência finita de símbolos do alfabeto justapostos.

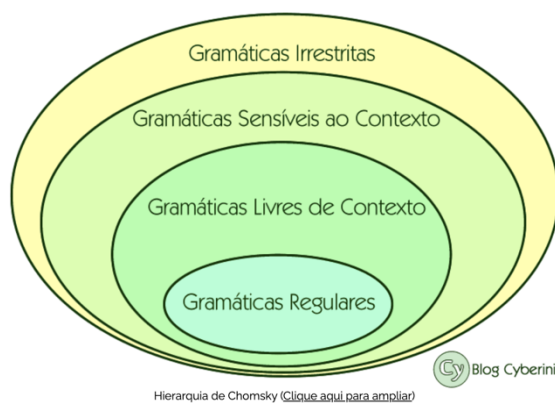
Forma sentencial: é uma sequência finita de símbolos de $\{N + T\}$ justapostos

Linguagem: é um conjunto de sentenças sobre um alfabeto.

4. Para que servem as gramáticas?

As gramáticas servem para definir as regras e especificar (definir) uma linguagem.

5. Quais diferentes tipos de gramáticas? O que os diferencia? Quais os componentes de uma gramática livre de contexto (GLC)?



Existem gramáticas livre de contexto e gramáticas regulares e as outras (que não tem interesse para a disciplina de compiladores: GSC e GI)

As gramáticas livres de contexto possuem uma única restrição onde deve haver um símbolo não terminal no lado esquerdo de cada regra de produção.

As gramáticas regulares possuem a mesma restrição do símbolo não terminal ao lado esquerdo das regras de produção, e também outra restrição no lado direito das regras de produção, onde ele deve ser composto apenas por uma cadeia vazia ou um símbolo terminal.

Os componentes de uma gramática livre de contexto são:

Conjunto de variáveis (NT), conjunto de símbolos terminais (T), conjunto de regras de produção (P) e símbolo inicial (S).

$G : (T, N, P, S)$

Gramática	Formato das produções	Reconhecedor
Regular	$N \rightarrow t$ $N \rightarrow t N$	Autômatos finitos
Livre de contexto	$N \rightarrow \alpha$, α pertence a $[T U N]^*$	Autômatos de pilha
Sensíveis ao contexto	$\alpha \rightarrow \beta$, $ \alpha $ menor ou igual a $ \beta $	Máquinas de Turing
Irrestritas	–	Máquinas de Turing

a. Exemplifique, utilizando as gramáticas abaixo

i. $G1: S \rightarrow (S) \mid \sim S \mid S \Theta S \mid p \mid q$

ii. $G2: CMD \rightarrow \text{if EXP then CMD}$
 $\mid \text{if EXP then CMD else C}$
 $\mid c$

$EXP \rightarrow e$

$CMD \rightarrow C \mid S$

$C \rightarrow c$

6. Que é uma linguagem $L(G)$?

É o conjunto de todas as sentenças de uma gramática G . É a linguagem gerada pela gramática G .

7. Que é uma derivação? O que significa dizer que uma gramática G *deriva* uma sentença “s”?

É o processo de se obter uma sentença “s” a partir do símbolo inicial de uma gramática G utilizando as regras da gramática.

Derivação é uma regra de reescrita de símbolos, onde trocamos o 'lado esquerdo' de uma produção por um de seus 'lados direitos' em uma forma sentencial. Notação: $\Rightarrow$

Redefinindo item 6:

$$L(G) = \{ p \mid S \Rightarrow^* p \}$$

Uma linguagem é o conjunto de todos os programas 'p' tal que S deriva p em 0 ou mais passos.

Compiladores... analisador sintático é o programa que verifica se p pertence a L(G).

8. O que é derivação mais a esquerda e mais a direita?

Significa obter um terminal utilizando as variáveis mais a esquerda ou a direita da palavra.

A derivação sempre seleciona o não terminal mais a (esquerda|direita) da forma sentencial para derivar.

a. Mostre cada uma delas para as para as sentenças abaixo

i. s1: $(\sim\sim p \Theta (p \Theta \sim q))$

$$\begin{aligned} \underline{S} &\Rightarrow^{lm} (\underline{S}) \\ &\Rightarrow^{lm} (\underline{S} \Theta S) \\ &\Rightarrow^{lm} (\sim \underline{S} \Theta S) \\ &\Rightarrow^{lm} (\sim\sim \underline{S} \Theta S) \\ &\Rightarrow^{lm} (\sim\sim p \Theta S) \\ &\Rightarrow^{lm} (\sim\sim p \Theta (S)) \\ &\Rightarrow^{lm} (\sim\sim p \Theta (S \Theta S)) \\ &\Rightarrow^{lm} (\sim\sim p \Theta (p \Theta S)) \\ &\Rightarrow^{lm} (\sim\sim p \Theta (p \Theta \sim S)) \\ &\Rightarrow^{lm} (\sim\sim p \Theta (p \Theta \sim q)) \end{aligned}$$

Logo s1 pertence a L(G1)

ii. s2: if e then if e then c else c

$$CMD \Rightarrow^{rm} \text{if EXP then } CMD$$

$$\Rightarrow^{rm} \text{if EXP then if EXP then CMD else CMD}$$

$$\Rightarrow^{rm} \text{if EXP then if EXP then CMD else c}$$

$$\Rightarrow^{rm} \text{if EXP then if EXP then c else c}$$

$$\Rightarrow^{rm} \text{if EXP then if e then c else c}$$

$$\Rightarrow^{rm} \text{if e then if e then c else c}$$

9. Que é árvore de derivação?

É uma forma de obter uma palavra utilizando as regras da linguagem.

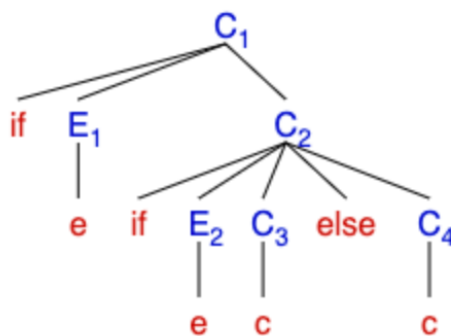
==> é uma representação gráfica do processo de derivação.

Se existe AD então s pertence a $L(G)$ (em compiladores... o programa está sintaticamente correto)

Obs. AD não contém informação sobre a ordem de escolha dos NT que foram derivados (esquerda|direita)

AD

1. Raiz é o símbolo inicial de G
2. Se N tem filhos $N_1, N_2, \dots, N_i$ então $N \rightarrow N_1 N_2 \dots N_i$ pertence a P
3. Nada mais é AD



a. Mostre as árvores associadas a cada uma das derivações do item 6

10. Quando uma gramática é ambígua? Qual o problema de trabalhar com gramáticas ambíguas? Pode-se dizer se G1 e G2 (item 3) são, ou não ambíguas? Como?

Uma gramática é dita ambígua quando **há mais de uma forma de derivar** a mesma palavra na linguagem. A gramática G1 é ambígua pois a palavra "p S q" pode ser derivado de duas formas: $S \rightarrow S \Theta S \rightarrow p \Theta S \rightarrow p \Theta q$ e $S \rightarrow S \Theta S \rightarrow S \Theta p \rightarrow p \Theta p$. A G2 também é ambígua.

Uma gramática é dita ambígua quando existir mais de uma AD para um mesmo programa (string, sentença)

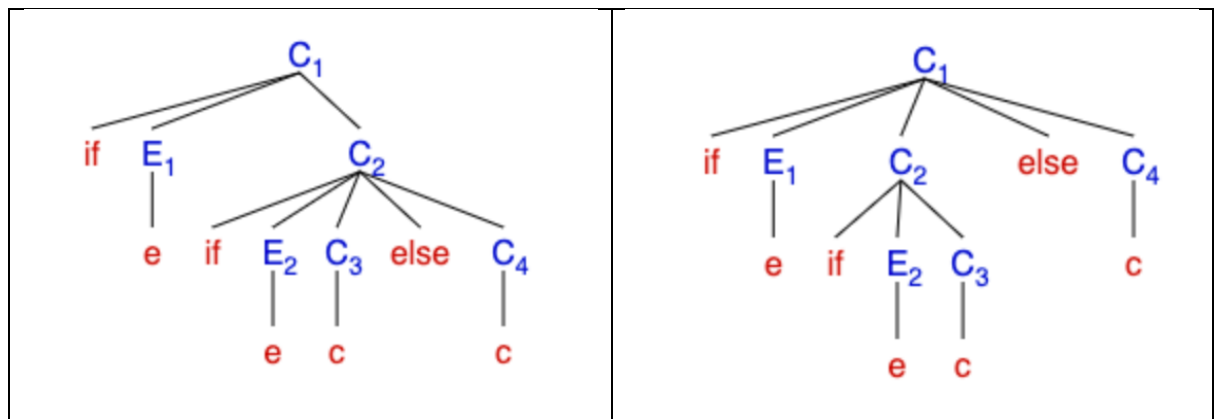
Problema: cada árvore associa um significado diferente ao programa

```
P1: if (a > b)
    if ( a > c)
        Print a;
    else
        Print b;
```

$C \Rightarrow \text{if } E \ C$

$C = \text{if } E \ C \ \text{else } C$

(errada para linguagens derivadas de C)



11. O que significa dizer que um operador tem precedência sobre outro? E dizer que o operador é associativo a esquerda, a direita ou que ele é não associativo? (esta não é de LF, mas é possível responder...). É possível determinar a associatividade e precedência dos operadores nas gramáticas G3 e G4 abaixo?

Um operador tem precedência sobre o outro quando operações contendo o primeiro devem ser resolvidas antes de operações contendo o segundo. Quando um operador é associativo à esquerda e temos uma sequência de operações desse operador, tais operações devem ser executadas da esquerda para a direita, ou da direita para a esquerda caso a associatividade seja à direita.

$$R1 = (- b + \text{sqrt}(\text{delta})) / (2 * a) ;$$

() - divisão tem precedência sobre a soma e preciso somar antes de dividir

() - divisão e multiplicação em a mesma precedência e são associativos a esquerda, precisamos garantir que multiplicação seja feita antes

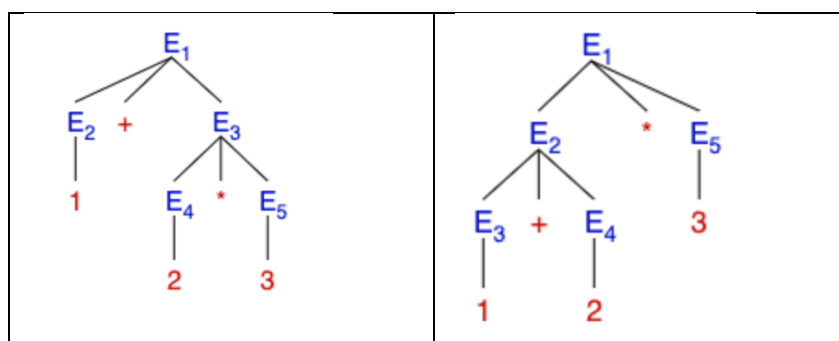
- a. G3: $E \rightarrow E + E \mid E - E \mid E * E \mid (E) \mid \text{numero}$

Gramática é ambígua, precedência e associatividade dependem da árvore gerada.

P: $1 + 2 * 3$

$$E \Rightarrow E + E$$

$$E \Rightarrow E * E$$



P2: 8 / 4 / 2

'Significado de associatividade a esquerda: 1'	'Significado de associatividade a direita: 4'
<pre> graph TD E1[E1] --- E2[E2] E1 --- S1[/] E1 --- E5[E5] E2 --- E3[E3] E2 --- S2[/] E2 --- E4[E4] E3 --- 8[8] E4 --- 4[4] E5 --- 2[2] </pre>	<pre> graph TD E1[E1] --- E2[E2] E1 --- S1[/] E1 --- E3[E3] E2 --- 8[8] E3 --- E4[E4] E3 --- S2[/] E3 --- E5[E5] E4 --- 4[4] E5 --- 2[2] </pre>

Nesse caso, todos os operadores possuem a mesma precedência.

- b. $ER \rightarrow EL > EL \mid EL \quad a > b \ \&\& \ b > c$, não aceita $a > b > c$
- $EL \rightarrow EL \text{ AND } E \mid E$
- $E \rightarrow E + T$
- $\mid E - T$
- $\mid T$
- $T \rightarrow T * F$
- $\mid T \% F$
- $\mid T / F$
- $\mid F$
- $F \rightarrow (E)$
- $\mid \text{numero}$
- $\mid \text{ident}$

Gramática não ambígua para operadores de soma e multiplicação. '*' tem precedência sobre a '+' e ambos são associativos à esquerda

Algoritmo:

Um conjunto de produções por nível de precedência (maior precedência mais 'abaixo')

Associatividade é dada da forma:

- Recursão à esquerda – associatividade à esquerda
- Recursão à direita – associatividade à direita
- Sem recursão - operador não associativo, pe: java não aceita $a > b > c$

12. O que é fatoração a esquerda? E recursão a esquerda? Realize a fatoração a esquerda e eliminação da recursão a esquerda das gramáticas G1, G2, G3 e G4, quando for possível.

- Será visto junto com Análise Sintática Descendente

13. Qual a diferença entre um autômatos (finitos, de pilha) e gramáticas?

Um autômato **reconhece** sentenças que pertencem a uma determinada linguagem.

Uma gramática **gera** sentenças que pertencem a uma determinada linguagem.

AF - ER ==> análise léxica

PDA - GLC ==> análise sintática

MT - GSC/GI ==> inúteis para compiladores